GROUP
6



MINI PROJECT
BẦU CUA TÔM CÁ

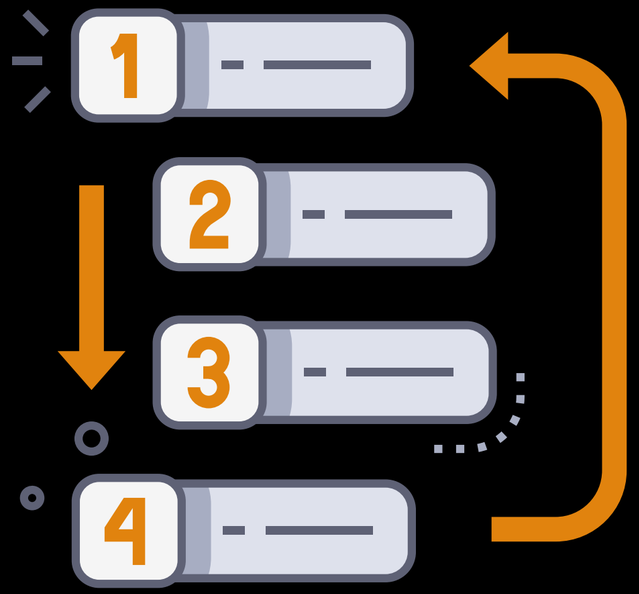
Who gonna win
Which is the best strategy



THÀNH VIÊN NHÓM

Mã số sinh viên	Họ và tên
25521137	Trần Ngô Nhật Minh
25521575	Lê Thanh Sang
25521423	Lê Hữu Phúc
25520188	Nguyễn Hoàng Bảo Châu
25520524	Lê Thị Thu Hiền

MỤC LỤC



1	Mục tiêu đồ án
2	Phạm vi và công cụ thực hiện
3	Mô hình lý thuyết và cơ sở toán học
4	Thiết kế kiến trúc và lập trình hệ thống
5	Triển khai mô phỏng và kết quả
6	Kết luận và hướng phát triển

MỤC TIÊU KỸ THUẬT & LẬP TRÌNH C++

Kiến trúc OOP

- Xây dựng hệ thống phân cấp rõ ràng: Player, House, BettingStrategy. Áp dụng mẫu Design Patterns chuẩn mực.

Tối ưu đa luồng

- Triển khai std::thread để thực thi hàng triệu ván chơi đồng thời, tận dụng tối đa tài nguyên CPU phần cứng.

RNG độc lập

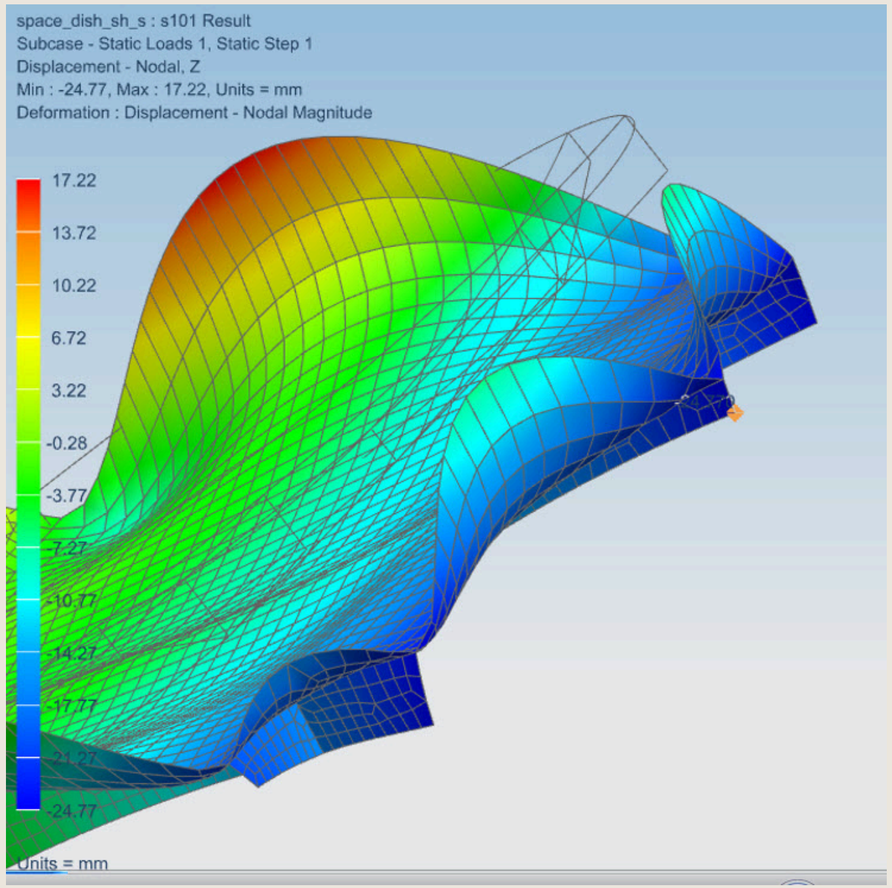
- Cơ chế thread_local sinh số ngẫu nhiên riêng biệt cho từng luồng, đảm bảo tính chính xác và tránh xung đột.

Quản lý bộ nhớ

- Ứng dụng RAII & Smart Pointers đảm bảo hệ thống không rò rỉ tài nguyên trong các kịch bản Big Data.

MỤC TIÊU THỐNG KÊ TOÁN HỌC

- **Monte Carlo:** Mô phỏng số lượng lớn ván chơi để kiểm chứng các định luật xác suất thực nghiệm.
- **Xác định EV:** Tính toán giá trị kỳ vọng âm, phơi bày lợi thế toán học không thể vượt qua của nhà cái.
- **Gambler’s Ruin:** Mô phỏng quá trình phá sản tất yếu và hiện tượng "càng chơi càng thua" theo thời gian.
- **So sánh Chiến thuật:** Đánh giá hiệu quả thực tế giữa Martingale (Gấp thếp) và Spread Betting (Rải thảm).



CÔNG NGHỆ VÀ CÔNG CỤ SỬ DỤNG

Hạng mục	Công nghệ và chi tiết kỹ thuật
Ngôn ngữ chính	**C++ 17:** Sử dụng Smart Pointers, RAII, và High-performance STL.
Xử lý số liệu	**Multithreading:** Lock-free design, std::thread, hardware_concurrency.
Mẫu thiết kế	**Design Patterns:** Strategy (Chiến thuật) và Factory (Khởi tạo đối tượng).
Phân tích & UI	**Python Data Stack:** Pandas (xử lý), Matplotlib/Numpy (trực quan hóa).
Môi trường & Data	**Visual Studio 2022;** Định dạng CSV cho dữ liệu thô Big Data.

GAMBLER’S RUIN

- Bản chất: Mô tả kịch bản một người chơi có số vốn hữu hạn đối đầu với một hệ thống (nhà cái) có số vốn gần như vô hạn trong một trò chơi có kỳ vọng âm.
- Kết luận: Nếu chơi đủ lâu, xác suất bạn trắng tay là 100%. Đây là định mệnh toán học, không phải do đen đủi

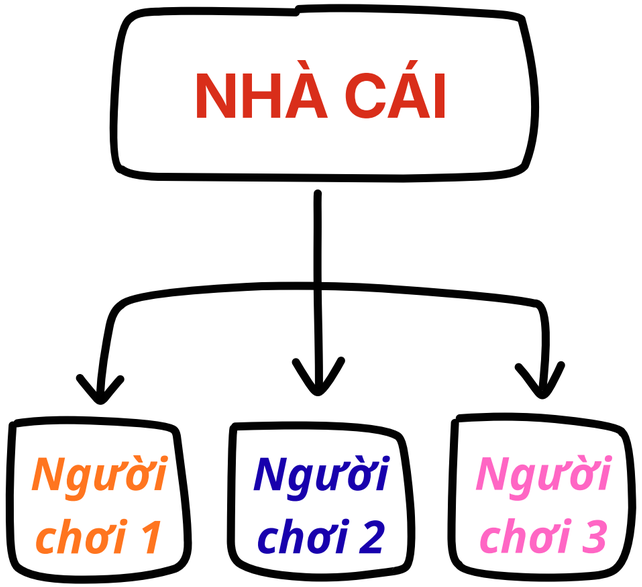
MONTE CARLO

- Thay vì ngồi giải những công thức xác suất tổ hợp khô khan, phương pháp này sử dụng sức mạnh máy tính để thực hiện hàng triệu mô phỏng ngẫu nhiên nhằm tìm ra quy luật.
- Vạch trần sự biến động của túi tiền và chứng minh các "chiến thuật" thường chỉ mang lại cảm giác thắng giả tạo trước khi "cháy túi".

LUẬT CHƠI BẦU CUA TÔM CÁ







THUA >>>>

X1

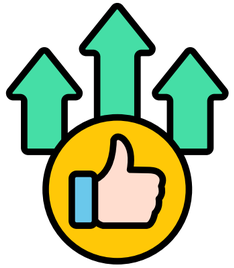
X2

X3

GAME ON

LỢI THẾ NHÀ CÁI

Tính toán House Edge và Kỳ vọng (EV)



7,87%

Nếu một người chơi đặt cược 1 đơn vị vào một biểu tượng duy nhất (ví dụ: "Bầu"), không gian xác suất toán học sẽ là:

- Thua: $(5/6)^3 = 125/216 \approx 57,87$
- Thắng 1: $\binom{3}{1} \times (1/6) \times (5/6)^2 = 75/216 \approx 34,72\%$
- Thắng 2: $\binom{3}{2} \times (1/6)^2 \times (5/6) = 15/216 \approx 6,94\%$
- Thắng 3: $\binom{3}{3} \times (1/6)^3 \times (5/6)^0 = 1/216 \approx 0,46\%$

Tính toán kỳ vọng EV:

$$EV = \left(-1 \times \frac{125}{216}\right) + \left(1 \times \frac{75}{216}\right) + \left(2 \times \frac{15}{216}\right) + \left(3 \times \frac{1}{216}\right)$$

$$EV = \frac{-125 + 75 + 30 + 3}{216} = \frac{-17}{216} \approx -0,0787$$

CÁC CHIẾN THUẬT ĐẶT CƯỢC



Chiến thuật chung thủy

Martingale-Gấp thếp 1 cửa

- Mục tiêu: Dùng duy nhất một ván thắng để gỡ toàn bộ vốn đã mất và thu về lợi nhuận bằng đúng mức cược ban đầu.
- Cách chơi:
- Chọn một cửa (linh vật) cố định.
- Thua: Gấp đôi tiền cược ở ván sau (1->2->4->8)
- Thắng: Quay lại mức cược khởi điểm.

Chiến thuật rải thảm

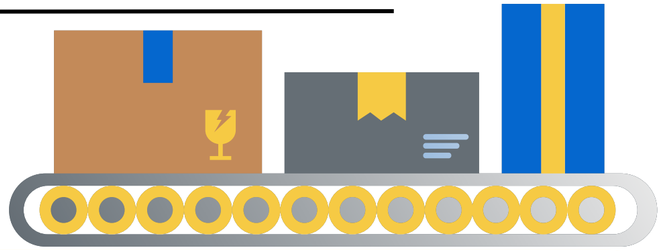
Spread Betting-Phân bố rủi ro



- Mục tiêu: Tăng tần suất thắng (Hit Frequency). Thay vì chỉ có 42,13% cơ hội thắng khi đặt 1 cửa, việc rải nhiều cửa giúp bạn gần như ván nào cũng có cảm giác "trúng" một cái gì đó.
- Ví dụ: Bạn đặt 1 đơn vị vào Bầu, 1 vào Cua, và 1 vào Cá. Tổng cược là 3 đơn vị.

KIẾN TRÚC TỔNG THỂ

System Architecture



Phân xưởng 1: Trạm Mô phỏng Tốc độ cao (Viết bằng C++)

- Người Quản đốc (File main.cpp)
- Bàn Chơi Game (đối tượng) (File Simulator.cpp & Simulator.h)
- Các Thực Thể Tham Gia (đối tượng) (File House.h & Player.h)
- Bộ Não Chiến Thuật (File BettingStrategy.h & Strategies.h)
- Bộ khởi tạo người chơi (File PlayerFactory.h)

Phân xưởng 2: Kho Lưu Trữ Hồ Sơ (Các file CSV)

Phân xưởng 3: Phòng Tổng Hợp & Đồ Họa (Viết bằng Python)

- Bộ phận Thu thập (Data Aggregator)
- Bộ phận Chỉnh lý Dữ liệu (Xử lý Mảng)
- Bộ phận In Ấn Biểu Đồ (File bao_cao_nhacai_*.py)

KIẾN TRÚC TỔNG THỂ

System Architecture

Khởi tạo & phân bố tài nguyên

- Đếm số luồng CPU
- Chia đôi luồng (4/4)

01





02

Chạy đa luồng song song

- Luồng 1...4: Nhà cái nhỏ
- Luồng 5...8: Nhà cái lớn



Điểm hội tụ (Thread Join)

Chờ tất cả các luồng ghi xong file CSV

03



04

Gọi Python Scripts

- Chạy bao_cao_nhacai_nho.py
- Chạy bao_cao_nhacai_lon.py



Xử lý dữ liệu (Pandas&Numpy)

- Đọc tất cả các file CSV
- Đồng bộ dữ liệu
- Tính toán kỳ vọng (EV)

05



06

Vẽ biểu đồ

- Vẽ chart luồng kết thúc nhanh nhất
- Vẽ chart luồng trụ lại lâu nhất
- Vẽ chart kỳ vọng trung bình

CHI TIẾT BÊN TRONG MỖI LUỒNG

1

Tạo nhà cái (Vốn nhỏ hoặc vốn lớn)



2

Tạo hai người chơi (Chung thủy và Rải thảm): vốn 800k



3

Bắt đầu vòng lặp ván cược: 1 → max

- Tung 3 viên xúc xắc ngẫu nhiên
- Người chơi tính toán tiền cược dựa trên chiến thuật
- Đối chiếu kết quả: nhà cái thu/trả tiền
- Cập nhật số dư tiền của nhà cái và 2 người chơi
- Ghi tạm kết quả vào bộ đệm buffer → Đủ 10000 ván → xả bộ nhớ
- Nếu số tiền nhà cái =0 hoặc người chơi =0 thì STOP

LẬP TRÌNH HƯỚNG ĐỐI TƯỢNG

OOP Design Pattern

Strategy Pattern: Linh hoạt hóa logic đặt cược

File BettingStrategy.h định nghĩa lớp trừu tượng (abstract class) cho các chiến lược cược

```cpp
#pragma once
#include <map>

enum class Symbol { BAU, CUA, TOM, CA, GA, NAI };

class BettingStrategy {
public:
    virtual ~BettingStrategy() = default;
    virtual std::map<Symbol, double> calculateBets(double balance, double baseBet, int losses) const = 0;
};
```

LẬP TRÌNH HƯỚNG ĐỐI TƯỢNG

OOP Design Pattern

Strategy Pattern: Linh hoạt hóa logic đặt cược

File Strategies.h cài đặt cụ thể các chiến lược như Martingale (gấp thếp) và Spread Betting (rải thảm)

```cpp
#include <cmath>
#include <algorithm>
#include <random>
#include <vector>

class ChungThuyStrategy : public BettingStrategy {
public:
    std::map<Symbol, double> calculateBets(double balance, double baseBet, int losses) const override {
        std::map<Symbol, double> bets;
        double amount = baseBet * std::pow(2, losses);
        amount = std::min(amount, balance);

        if (amount > 0) {
            static thread_local std::mt19937 rng(std::random_device{}());
            std::uniform_int_distribution<int> dist(0, 5);

            Symbol randomSymbol = static_cast<Symbol>(dist(rng));
            bets[randomSymbol] = amount;
        }
        return bets;
    }
};
```

LẬP TRÌNH HƯỚNG ĐỐI TƯỢNG

OOP Design Pattern

Strategy Pattern: Linh hoạt hóa logic đặt cược

File Strategies.h cài đặt cụ thể các chiến lược như Martingale (gấp thếp) và Spread Betting (rải thảm)

```
26  class RaiThamStrategy : public BettingStrategy {
27  public:
28      std::map<Symbol, double> calculateBets(double balance, double baseBet, int losses) const override {
29          std::map<Symbol, double> bets;
30          double splitBet = std::min(baseBet, balance / 3.0);
31          if (splitBet > 0) {
32              static thread_local std::mt19937 rng(std::random_device{}());
33
34              std::vector<Symbol> allSymbols = {
35                  Symbol::BAU, Symbol::CUA, Symbol::TOM,
36                  Symbol::CA, Symbol::GA, Symbol::NAI
37              };
38
39              std::shuffle(allSymbols.begin(), allSymbols.end(), rng);
40
41              bets[allSymbols[0]] = splitBet;
42              bets[allSymbols[1]] = splitBet;
43              bets[allSymbols[2]] = splitBet;
44          }
45          return bets;
46      }
47  };
```

LẬP TRÌNH HƯỚNG ĐỐI TƯỢNG

OOP Design Pattern

Factory Pattern: Quản lý khởi tạo người chơi

```cpp
#pragma once
#include "Player.h"
#include "Strategies.h"
#include <memory>
#include <string>

class PlayerFactory {
public:
    enum class Type { CHUNG_THUY, RAI_THAM };

    static std::unique_ptr<Player> createPlayer(Type type, const std::string& name, double balance) {
        double defaultBaseBet = 100.0;
        std::unique_ptr<BettingStrategy> strat;

        switch (type) {
        case Type::CHUNG_THUY:
            strat = std::make_unique<ChungThuyStrategy>(); break;
        case Type::RAI_THAM:
            strat = std::make_unique<RaiThamStrategy>(); break;
        }
        return std::make_unique<Player>(name, balance, defaultBaseBet, std::move(strat));
    }
};
```

QUẢN LÝ BỘ NHỚ HIỆN ĐẠI

Modern Memory Management

Quản lý tài nguyên đa hình (Polymorphism) trong Class

```cpp
class Player {
private:
    std::string name;
    double balance;
    double baseBet;
    int consecutiveLosses = 0;
    std::unique_ptr<BettingStrategy> strategy;
```

```cpp
class Simulator {
private:
    int numSimulations;
    std::string outputFileName;
    std::vector<std::unique_ptr<Player>> players;
    std::unique_ptr<House> house;

public:
    Simulator(int nums, const std::string& fileName, std::unique_ptr<House> h)
        : numSimulations(nums), outputFileName(fileName), house(std::move(h)) {
    }

    void addPlayer(std::unique_ptr<Player> p);
    void run();
};
```

QUẢN LÝ BỘ NHỚ HIỆN ĐẠI

Modern Memory Management

Khởi tạo an toàn kết hợp với Factory Pattern

```
class PlayerFactory {
public:
    enum class Type { CHUNG_THUY, RAI_THAM };

    static std::unique_ptr<Player> createPlayer(Type type, const std::string& name, double balance) {
        double defaultBaseBet = 100.0;
        std::unique_ptr<BettingStrategy> strat;

        switch (type) {
        case Type::CHUNG_THUY:
            strat = std::make_unique<ChungThuyStrategy>(); break;
        case Type::RAI_THAM:
            strat = std::make_unique<RaiThamStrategy>(); break;
        }
        return std::make_unique<Player>(name, balance, defaultBaseBet, std::move(strat));
    }
};
```

- Trong PlayerFactory, std::make_unique được sử dụng để khởi tạo các đối tượng và trả về std::unique_ptr<Player>.
- Việc này tuân thủ nguyên tắc cấp phát tài nguyên an toàn, ngăn chặn việc rò rỉ bộ nhớ (memory leak) ngay từ lúc khởi tạo.

QUẢN LÝ BỘ NHỚ HIỆN ĐẠI

Modern Memory Management

Vận hành luồng dữ liệu (Data Flow)

```
void runSimulationNode(int threadID, int numSimulations, double houseBalance, const std::string& prefix) {
    std::string fileName = prefix + "_" + std::to_string(threadID) + ".csv";

    auto house = std::make_unique<House>("NhaCai", houseBalance);
    Simulator sim(numSimulations, fileName, std::move(house));

    sim.addPlayer(PlayerFactory::createPlayer(PlayerFactory::Type::CHUNG_THUY, "ChungThuy", 800000.0));
    sim.addPlayer(PlayerFactory::createPlayer(PlayerFactory::Type::RAI_THAM, "RaiTham", 800000.0));

    sim.run();
}
```

- Ủy quyền sở hữu: std::unique_ptr khởi tạo House và Player, sau đó chuyển giao toàn bộ quyền quản lý cho biến cục bộ sim.
- Tự động giải phóng: Khi hàm kết thúc, biến sim đi ra khỏi tầm vực (out of scope), tự động kích hoạt dọn dẹp toàn bộ tài nguyên con bên trong nó một cách an toàn và triệt để.

KỸ THUẬT LẬP TRÌNH ĐA LUỒNG (MULTI-THREADING)

```cpp
unsigned int numThreads = std::thread::hardware_concurrency();
if (numThreads < 2) numThreads = 4;

unsigned int halfThreads = numThreads / 2;
int simsPerThread = totalSimulations / halfThreads;

std::cout << "================================================\n";
std::cout << " MO PHONG DA LUONG TICH HOP\n";
std::cout << "================================================\n";
std::cout << "[-] Phat hien CPU co " << numThreads << " luong.\n";
std::cout << "[-] Chia " << halfThreads << " luong cho Nha Cai NHO \n";
std::cout << "[-] Chia " << halfThreads << " luong cho Nha Cai LON \n";
std::cout << "[!] Dang chay song song... Vui long doi!\n";

auto start_time = std::chrono::high_resolution_clock::now();

std::vector<std::thread> workers;

for (unsigned int i = 1; i <= halfThreads; ++i) {
    workers.push_back(std::thread(runSimulationNode, i, simsPerThread, 15000.0, "data_nho"));
}

for (unsigned int i = 1; i <= halfThreads; ++i) {
    workers.push_back(std::thread(runSimulationNode, i, simsPerThread, 6000000.0, "data_lon"));
}

for (auto& t : workers) {
    t.join();
}
```



KHỞI TẠO LUỒNG ĐỘNG

KỸ THUẬT LẬP TRÌNH ĐA LUỒNG (MULTI-THREADING)

```
    if (amount > 0) {
        static thread_local std::mt19937 rng(std::random_device{}());
        std::uniform_int_distribution<int> dist(0, 5);

        Symbol randomSymbol = static_cast<Symbol>(dist(rng));
        bets[randomSymbol] = amount;
    }
```

```
if (splitBet > 0) {
    static thread_local std::mt19937 rng(std::random_device{}());

    std::vector<Symbol> allSymbols = {
        Symbol::BAU, Symbol::CUA, Symbol::TOM,
        Symbol::CA, Symbol::GA, Symbol::NAI
    };

    std::shuffle(allSymbols.begin(), allSymbols.end(), rng);
```



ĐẢM BẢO TÍNH NGẪU NHIÊN

TỐI ƯU HÓA HIỆU NĂNG I/O

I/O Performance Optimization

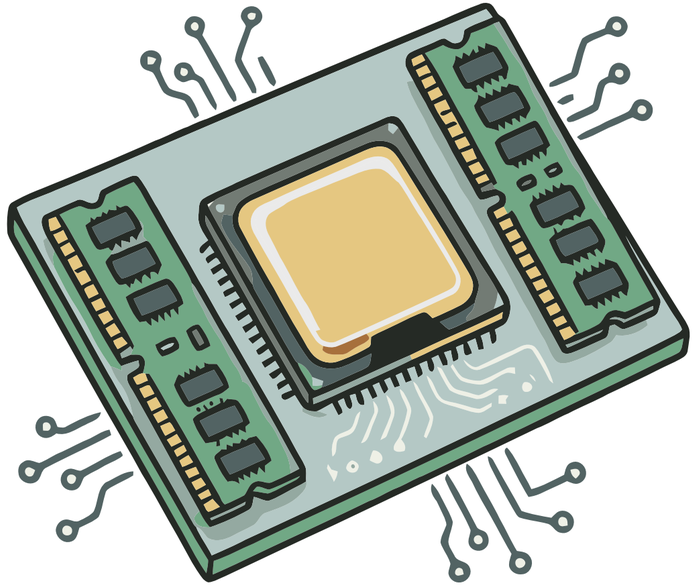
Kỹ thuật Ghi đệm RAM
(RAM Buffering) bằng std::ostringstream

Giải quyết nút thắt cổ chai ổ cứng
(Disk I/O Bottleneck)

KỸ THUẬT GHI ĐỆM BẰNG RAM
RAM BUFFERING

```
std::ostringstream buffer;

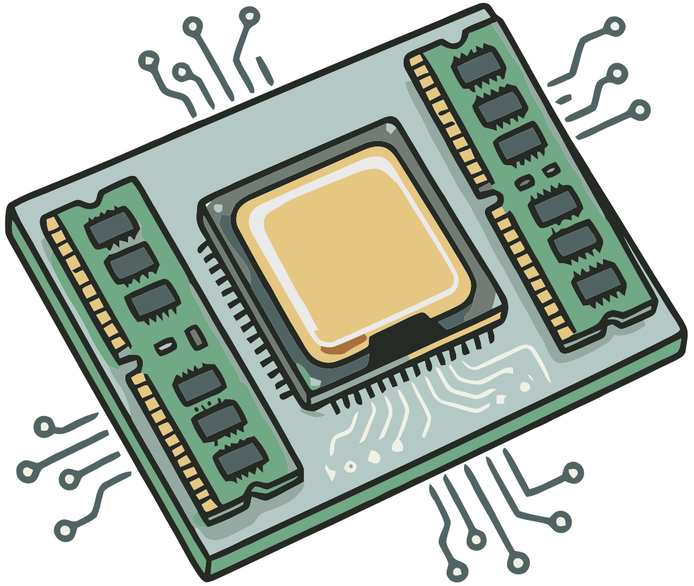
for (int i = 1; i <= numSimulations; ++i) {
    std::map<Symbol, int> diceCounts;
    for (int d = 0; d < 3; ++d) {
        Symbol result = static_cast<Symbol>(dist(rng));
        diceCounts[result]++;
    }
    bool allPlayersBankrupt = true;
    buffer << i;
```



Thay vì ghi chậm chạp từng dòng trực tiếp xuống ổ cứng, chương trình nối (<<) dữ liệu vào bộ đệm RAM (std::ostringstream buffer) để tận dụng tốc độ xử lý siêu tốc của bộ nhớ trong.

GIẢI QUYẾT NÚT THẮT CỔ CHAI Ổ CỨNG DISK I/O BOTTLENECK

```
    if (i % 10000 == 0) {
        outFile << buffer.str();
        buffer.str("");
        buffer.clear();
    }
    if (allPlayersBankrupt || house->isBankrupt()) break;
}
outFile << buffer.str();
outFile.close();
```



Để tránh "nút thắt cổ chai" do tốc độ ghi ổ cứng (Disk I/O) quá chậm, chương trình gom đủ 10.000 dòng dữ liệu vào bộ đệm rồi mới "xả" (flush) xuống đĩa cùng một lúc, thay vì ghi lắt nhắt từng dòng.

TÍCH HỢP PYTHON SINH BIỂU ĐỒ TỰ ĐỘNG BẰNG C++

- Sử dụng hàm std::system() để gọi trực tiếp các tệp script Python (bao_cao_nhacai_nho.py và bao_cao_nhacai_lon.py), tạo thành một "đường ống dữ liệu" (data pipeline)
- Ở phía Python, các tệp script này sẽ tự động đọc dữ liệu từ thư mục data bằng thư viện glob và vẽ biểu đồ bằng matplotlib

```cpp
auto end_time = std::chrono::high_resolution_clock::now();
std::chrono::duration<double> elapsed = end_time - start_time;

std::cout << "------------------------------------------------\n";
std::cout << "[+] Mo phong hoan tat 100%!\n";
std::cout << "[+] Thoi gian chay thuc te: " << elapsed.count() << " giay.\n";
std::cout << "------------------------------------------------\n";
std::cout << "Dang khoi chay he thong bieu do Python...\n";

std::system("python scripts/bao_cao_nhacai_nho.py");
std::system("python scripts/bao_cao_nhacai_lon.py");
```

```python
import pandas as pd
import matplotlib.pyplot as plt
import glob
import os
import numpy as np

print("\n" + "="*50)
print(" KỊCH BẢN 2: BẦU CUA - NHÀ CÁI VỐN LỚN")
print("="*50)

script_dir = os.path.dirname(os.path.abspath(__file__))
data_dir = os.path.join(script_dir, '..', 'data')
file_list = glob.glob(os.path.join(data_dir, 'data_lon_*.csv'))
```

Thiết lập các thông số kịch bản mô phỏng

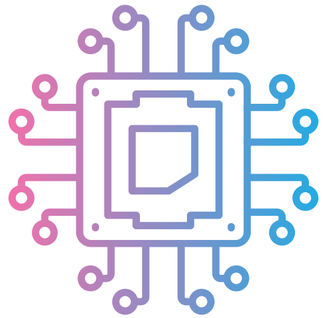
Số vòng mô phỏng	Thiết lập chạy 100.000.000 vòng (100 triệu vòng).
Thiết lập đa luồng	• Số luồng được lấy tự động từ CPU • Chia đều cho 2 kịch bản: Nhà cái vốn nhỏ và Nhà cái vốn lớn → Giúp tối ưu hiệu năng và rút ngắn thời gian chạy mô phỏng
Thiết lập vốn ban đầu	• Nhà cái vốn nhỏ: dễ bị phá sản do biến động lớn • Nhà cái vốn lớn: có khả năng hấp thụ biến động và thể hiện rõ quy luật xác suất • Người chơi được cấp vốn ban đầu cố định và sử dụng các chiến lược cược khác nhau.
Thiết lập chiến lược cược	• Chung thủy (Martingale): gấp thếp khi thua • Rải thảm (Spread Betting): chia vốn cược nhiều cửa • Các người chơi được khởi tạo thông qua Factory Pattern.
Điều kiện dừng mô phỏng	• Người chơi phá sản (balance = 0) • Nhà cái phá sản • Hoặc đạt đủ số vòng mô phỏng

THIẾT LẬP CÁC THÔNG SỐ KỊCH BẢN MÔ PHỎNG

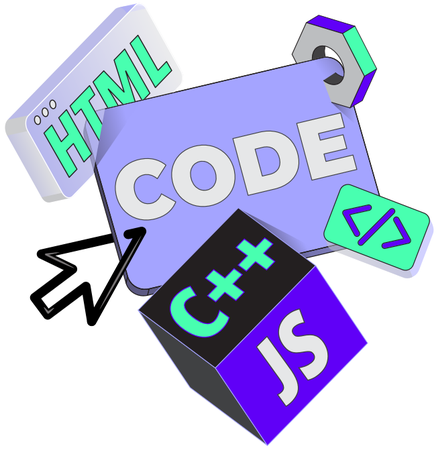
```cpp
int main() {
    int totalSimulations = 100000000;

    unsigned int numThreads = std::thread::hardware_concurrency();
    if (numThreads < 2) numThreads = 4;

    unsigned int halfThreads = numThreads / 2;
    int simsPerThread = totalSimulations / halfThreads;
```



Thiết lập đa luồng




```cpp
    auto house = std::make_unique<House>("NhaCai", houseBalance);
    Simulator sim(numSimulations, fileName, std::move(house));

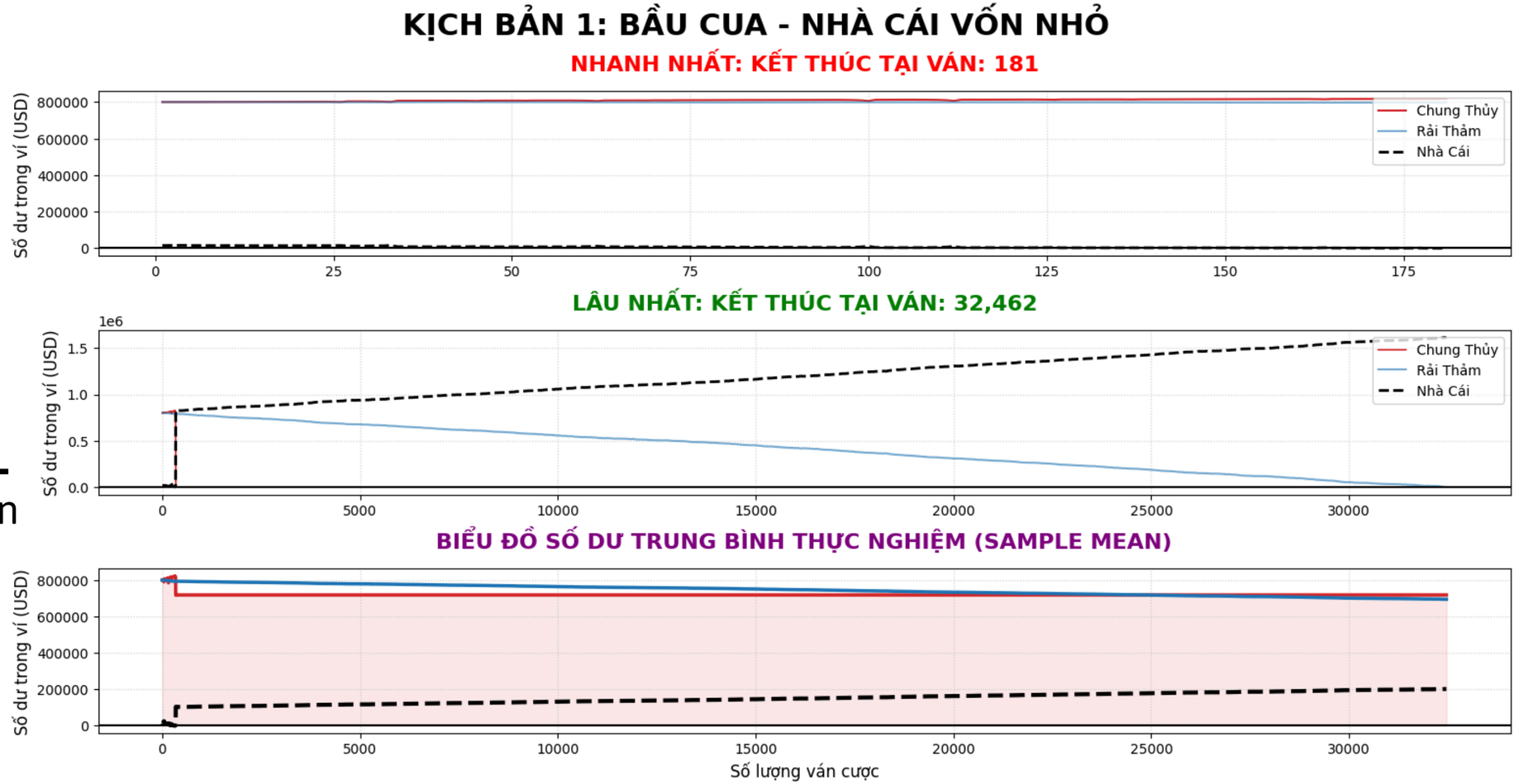
    sim.addPlayer(PlayerFactory::createPlayer(PlayerFactory::Type::CHUNG_THUY, "ChungThuy", 800000.0));
    sim.addPlayer(PlayerFactory::createPlayer(PlayerFactory::Type::RAI_THAM, "RaiTham", 800000.0));

    sim.run();
```

Thiết lập chiến lược thông qua Factory Pattern

NHÀ CÁI VỐN NHỎ

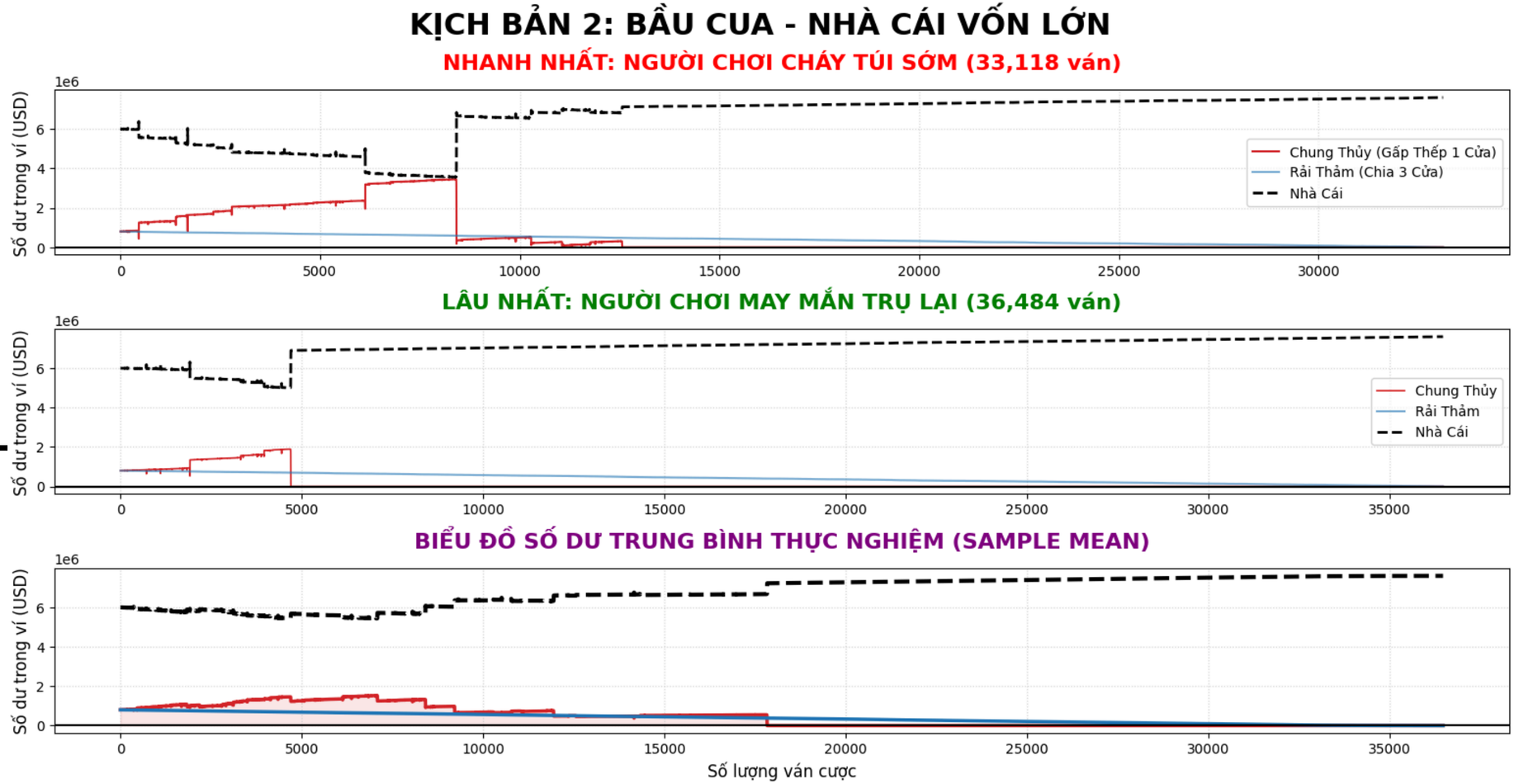
Mô phỏng Variance ngắn hạn



- Bản chất: Khi quỹ vốn của nhà cái còn nhỏ, Luật Số lớn vẫn chưa kịp phát huy tác dụng.
- Chiến thuật tấn công: Một người chơi sử dụng chiến thuật Martingale (Chung Thủy) một cách quyết liệt có thể tạo ra những cú sốc tài chính cực lớn cho hệ thống.
- Rủi ro phá sản: Như đã thấy trong biểu đồ "Nhanh nhất" (Fastest), nhà cái có thể bị phá sản chỉ trong vài giây nếu người chơi gặp được một chuỗi may mắn liên tục.

NHÀ CÁI VỐN LỚN

Quy luật số lớn



- Sự phá sản tất yếu của người chơi (Gambler's Ruin): Khi nhà cái có đủ vốn để hấp thụ mọi biến động, trò chơi sẽ chuyển thành một bài toán giới hạn toán học $(N \to \infty)$
- Ngay cả với số vốn khởi điểm lớn là 800.000 VNĐ, cả người chơi theo chiến thuật Rải Thảm (Spread) và Chung Thủy (Martingale) đều bị nghiền nát về mặt toán học bởi Lợi thế Nhà cái 7,87%. Đường cong Giá trị Kỳ vọng (EV) quyết định một cú lao dốc không ngừng nghỉ về mức 0.

ĐÁNH GIÁ CHUNG VỀ MỨC ĐỘ RỦI RO



Chiến thuật chung thủy

Martingale-Gấp thếp 1 cửa

- Tăng cược: Cấp số nhân sau mỗi lần thua.
- Nguy cơ: Cháy tài khoản vì chuỗi thua liên tiếp.
- Đánh giá: Rủi ro cao nhất trong các chiến thuật đặt cược.



Chiến thuật rải thảm

Spread Betting-Phân bố rủi ro

- Phân bổ: Đa dạng hóa các cửa đặt để giảm rủi ro.
- Biến động: Thấp, tâm lý người chơi ổn định hơn.
- Hậu quả: Không đảo ngược kết quả thua lỗ, chỉ làm chậm quá trình đi đến phá sản.

GÓC NHÌN CỦA NGƯỜI CHƠI

- **Kỳ Vọng Trung Bình (EV) của người chơi luôn là một đường cong cắm thẳng xuống 0**
- **"tối ưu nhất" không nằm ở việc tìm ra công thức chiến thắng tuyệt đối, mà là chiến lược quản trị rủi ro**



GÓC NHÌN CỦA NGƯỜI CHƠI

Tuyệt đối từ bỏ ảo tưởng Gấp thếp

Kỷ luật "Đánh nhanh rút gọn"

Sử dụng "Rải Thảm" làm khiên phòng



GÓC NHÌN CỦA NHÀ CÁI

1. Chuẩn bị nguồn Vốn dự phòng khổng lồ
2. Thiết lập Giới hạn cược trần (Max Bet / Table Limit)
3. Tối đa hóa số lượng ván cược (Volume)



KẾT QUẢ ĐẠT ĐƯỢC VỀ MẶT KỸ THUẬT PHẦN MỀM



Kiến trúc lai C++ & Python

Thiết lập quy trình Pipeline dữ liệu chuẩn xác. Sử dụng C++ làm động cơ lõi (Engine) cho tốc độ tính toán tối đa, kết hợp Python để xử lý mảng và trực quan hóa biểu đồ linh hoạt.

Mẫu thiết kế chuẩn mực

Áp dụng Strategy Pattern để tách biệt logic chiến thuật và Factory Pattern giúp tự động hóa khởi tạo. Hệ thống tuân thủ chặt chẽ OOP, đảm bảo tính dễ bảo trì và mở rộng.

Tối ưu đa luồng Lock-free

Thực thi 100.000.000 lượt mô phỏng song song không điểm nghẽn. Sử dụng hardware_concurrency() và thread_local để cách ly trạng thái ngẫu nhiên, bảo vệ tính độc lập Monte Carlo.

Quản lý bộ nhớ & Tối ưu I/O

Xóa bỏ nút thắt I/O bằng RAM Buffering (ghi batch 10k ván). Dùng unique_ptr loại bỏ hoàn toàn rò rỉ bộ nhớ (Zero Memory Leak).

KẾT QUẢ ĐẠT ĐƯỢC VỀ MẶT HỆ THỐNG



Luật số lớn

Trong dài hạn, bất kể chiến thuật nào, dòng tiền người chơi luôn hướng về 0 trong khi lợi nhuận nhà cái tăng trưởng ổn định theo Kỳ vọng (EV) âm.

Điểm yếu martingale

Chiến thuật Gấp thếp là đối tượng "cháy túi" nhanh nhất. Tốc độ tăng cược cấp số nhân nhanh chóng vắt kiệt số vốn hữu hạn của người chơi.

Rải thảm và biến động

Giúp nén biến động (Variance) và kéo dài sự tồn tại, nhưng không thể chiến thắng được quy luật số lớn. Dòng tiền bị rút cạn một cách "êm ái".

Hố đen thanh khoản

Nhà cái lớn ($6M) đóng vai trò hố đen hấp thụ mọi cú sốc xác suất, biến rủi ro ngẫu nhiên thành lợi nhuận định lượng ổn định.



HẠN CHẾ HIỆN TẠI CỦA HỆ THỐNG

- Thiếu Max Bet: Lỗ hổng phi thực tế khi không có giới hạn cược trần để cắt đứt chuỗi Gấp thếp của người chơi.
- Lãng phí CPU: Cấp phát std::vector và std::shuffle liên tục trong RaiThamStrategy gây chậm tốc độ mô phỏng.
- Nút thắt I/O: Xuất file CSV khổng lồ tiêu tốn băng thông và làm chậm quá trình gom nhóm dữ liệu của Python.
- Hành vi Tĩnh: Mô hình chưa thể hiện được các yếu tố tâm lý như lòng tham (FOMO), hoảng loạn hay bỏ cuộc sớm.
- Thuật ngữ EV: Đã điều chỉnh thành "Số dư trung bình thực nghiệm" để phản ánh đúng bản chất dữ liệu mô phỏng.



LỘ TRÌNH PHÁT TRIỂN TƯƠNG LAI



Bổ sung Business Logic thực tế

1

Tối ưu hóa Micro-performance

2

Đa dạng hóa hành vi con người

3

Tối ưu hóa Luồng Dữ liệu

4

Mở rộng Điện toán Phân tán

5

LỘ TRÌNH PHÁT TRIỂN TƯƠNG LAI



6

Đưa Trí tuệ Nhân tạo vào mô hình

7

Tổng quát hóa Engine Mô phỏng

8

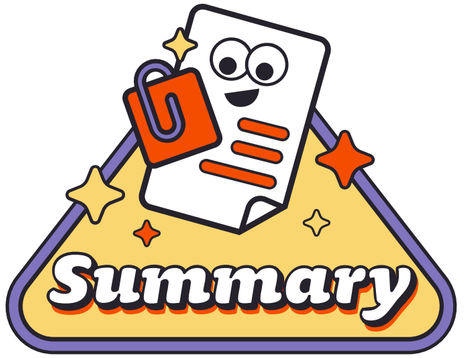
Nâng cấp Giao diện

9

Cải thiện độ chính xác thống kê của biểu đồ



TỔNG KẾT



Năng lực Thiết kế Phần mềm Hiện đại

Hệ thống hybrid hiệu năng cao:

- Kiến trúc: Strategy & Factory Patterns.
- Memory: Quản lý an toàn bằng RAII & unique_ptr.
- Performance: Đa luồng tối ưu hóa cho hàng trăm triệu ván cược.

Bản chất Thống kê & Sự thật Sòng bạc

- Kỳ vọng (EV): Âm (~ -7.87%), dẫn đến thua lỗ tất yếu.
- Martingale: Biến động cực lớn, gây phá sản nhanh chóng.
- Rải thảm: Ổn định dòng vốn nhưng không thay đổi kết cục dài hạn.

Kết luận & Giá trị Đạt được

- Kết hợp sức mạnh C++ và tính trực quan của Python.
- Chứng minh trực quan các định lý xác suất thông qua Dữ liệu lớn (Big Data).
- Nền tảng cho các hệ thống phân tích rủi ro phức tạp.

GROUP 6

Thank You



